Towards LSTM-Driven Forecasts for Next-Gen Water Level Monitoring

Lucas A. Raicu, Glenbrook South High School, USA

Daniel Grzenda, University of Chicago, USA

Kyle Chard, University of Chicago, USA

Abstract

Accurate forecasting of water levels is essential for tracking climate change and flood mitigation. Traditionally, predictions have been based on harmonic analysis and sensor networks maintained by the National Oceanographic and Atmospheric Administration. However, these methods struggle with increasingly erratic sea-level dynamics. To more accurately capture complex temporal dependencies, TidalMark leverages deep learning models, specifically Long-Short-Term Memory networks. These models, due to their ability to use gates to selectively retain or forget information over time, are able to learn long-range patterns in sequential time-series data, making them perfect to use for water-level predictions. Through extensive hyperparameter sweeps and comparisons across model variants, we have evaluated tradeoffs in accuracy, generalization (for time and architecture), and scalability. Our results show that properly tuned machine learning models consistently outperform the scientific-standard harmonic approaches between 2.1X and 4.7X (between 7D to 1D predictions) with the goal towards achieving adaptive, scalable, and accurate forecasting of coastal water levels.

1 Introduction

Coastal communities face increasing threats from flooding, sea-level rise, and extreme weather. [Service(2024)] Since the 1800s, harmonic analysis has been used to predict water levels by decomposing tides into cyclical components [Pugh(2004), Consoli et al.(2014b), Wikipedia contributors(2025b), Wikipedia contributors(2025c)]. The National Oceanographic and Atmospheric Administration (NOAA) [NOAA(2025)] maintains hundreds of sensors measuring coastal water levels and has adopted harmonic analysis to predict future water levels. While effective under stable conditions, it assumes linearity and stationarity (long-term sea-level rise), limiting accuracy under rapid environmental change [Consoli et al.(2014a)]. Figure 1 shows a major weather system over the course of several days that produced flooding water levels (yellow line) while NOAA predictions estimated water levels to be well in normal ranges.

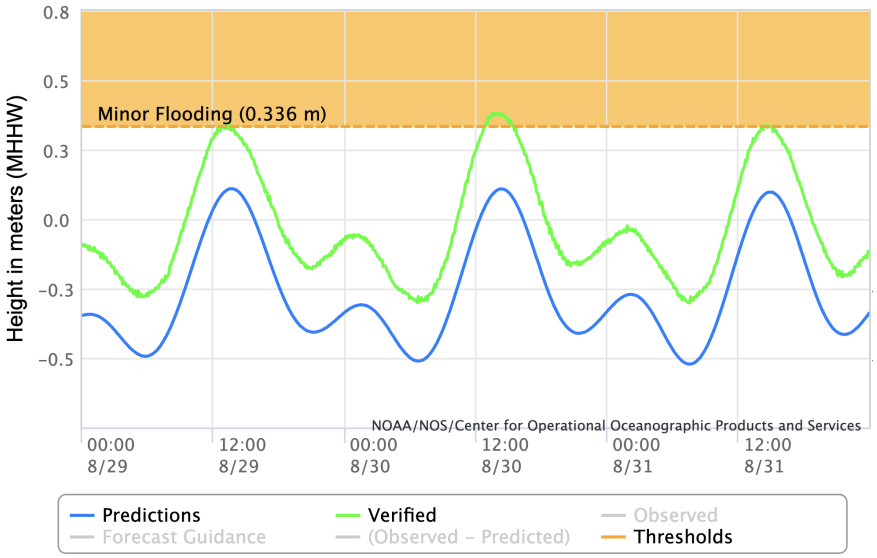
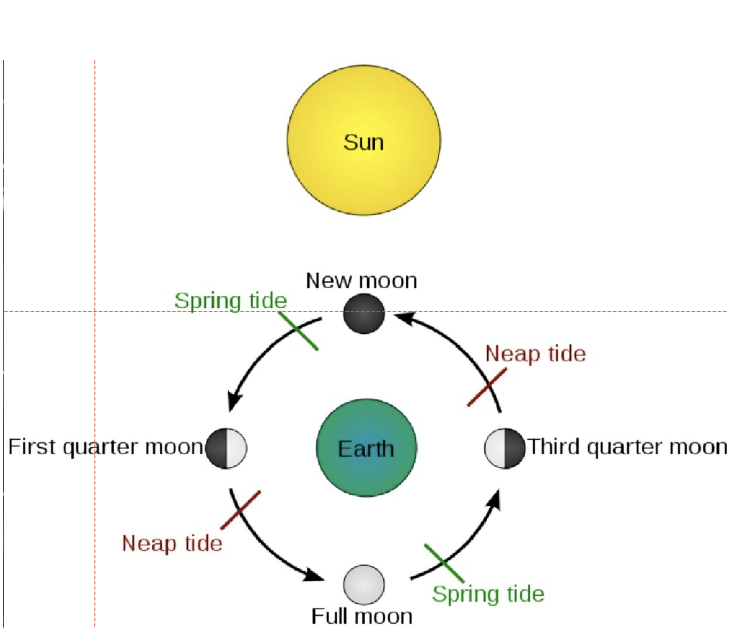
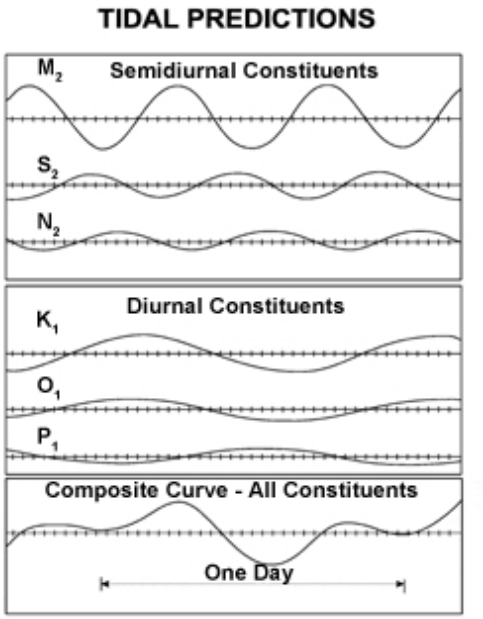


Figure 1: NOAA's predictions during non-periodic events; graphic obtained from NOAA National Water Level Observation Network (NWLON) [National Oceanic and Atmospheric Administration (NOAA)(2025)]

By design, harmonic analysis captures only astronomical drivers of tides (see Figure 2a) and uses those to decompose water-level signals into a fixed set of sinusoidal constituents, each with constant amplitude and period (see Figure 2b). However, this method does not account for wind, atmospheric pressure changes, or storms. Coastal systems exhibit non-stationary behavior (e.g. seasonal shifts) and nonlinear interactions (e.g. overtides and compound tides) among constituents, which harmonic analysis cannot adapt to dynamically. As climate change alters baseline sea levels, the fixed-parameter nature of harmonic analysis means that the constituents estimated from historical data will become increasingly unsuitable for future predictions.



(a) Harmonic analysis decomposes tidal forces into fixed-frequency constituents; graphic obtained from Perkins School for the Blind [Perkins School for the Blind(2022)]



(b) Tide modeling via additive synthesis of harmonics; graphic obtained from Wikipedia's Theory of tides [Wikipedia contributors(2025a)], [Wikimedia Commons contributors(2007)]

Figure 2: Gravitational Pull and Harmonic Analysis

In July 2025, there was a devastating flash flood in the Hill Country region of Central Texas, along the Guadalupe River near Kerr County [The Texas Tribune(2025)]. The river rose more than 20 feet in less than an hour, catching many people off guard and resulting in at least 100 confirmed deaths. This tragedy highlighted the urgent need for better water-level prediction systems. We wanted to understand why forecasting tools failed to issue warnings in time and how artificial intelligence could prevent similar disasters.

Recent advances in deep learning have demonstrated remarkable capability in modeling complex temporal dependencies within hydrological and environmental systems. Long Short-Term Memory (LSTM) networks [Hochreiter and Schmidhuber(1997a)] have been successfully applied to simulate rainfall-runoff dynamics and river discharge under variable climatic conditions [Kratzert et al.(2019)]. In Earth science more broadly, data-driven approaches have complemented physical process understanding, showing that neural architectures can bridge the gap between physical modeling and observational data [Reichstein et al.(2019)]. Many modern approaches such as LSTM-based models learn from both periodic and aperiodic patterns, and are therefore able to adapt to changing dynamics, fusing spatial context in ways that harmonic analysis alone cannot [Hochreiter and Schmidhuber(1997b)]. LSTMs are designed to retain context over long sequences by using

memory cells and gating mechanisms (input, forget, and output gates). While LSTM also features repetitive modules like traditional RNNs, LSTM incorporates several inter-connected layers This multi-layered design allows for complex long-term dependencies to be retained.

We propose to develop TidalMark, a system that leverages LSTM models to improve forecast lead times and adaptability for coastal water-level prediction. This research work's main contribution is that it identified Machine Learning models (LSTM) and its configuration parameters that can outperform prediction accuracy of traditional harmonic analysis between 2.1X and 4.7X (between 7D to 1D predictions). Figure 3 shows the histogram of all predictions using the LSTM compared to NOAA's predictions, showing the error size in meters.

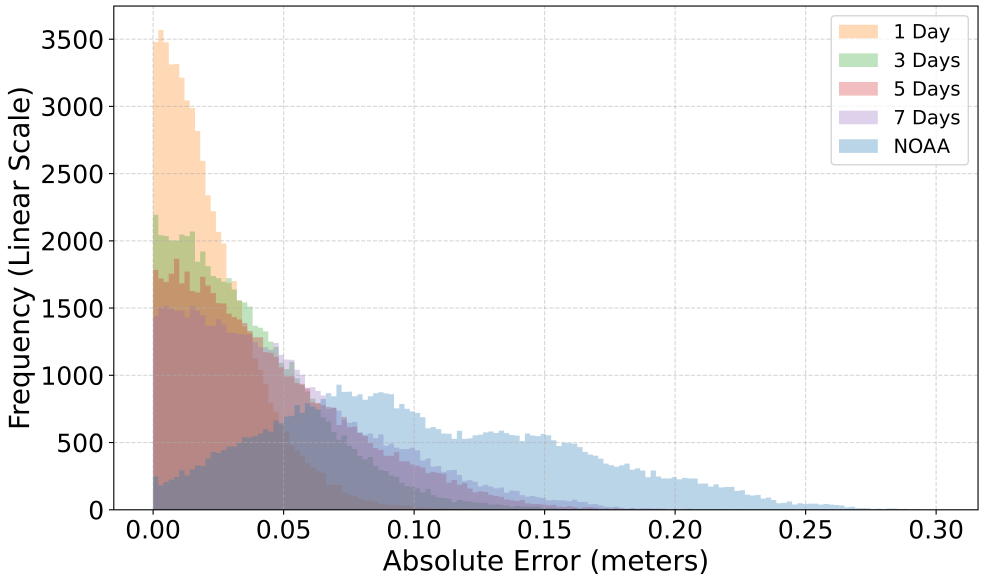


Figure 3: LSTM vs NOAA predictions at the Nawiliwili station, HI; figure is created by student with experimental data collected

2 Proposed Work

We implemented our LSTM models using PyTorch using the `nn.LSTM` module. Additional details on the source code are provided in Appendix A. We used sequential models to capture long-term-dependencies in water-levels. We performed a single-shot walk-forward validation. We split up our training data, validation data, and testing data in traditional 80:10:10 split. We trained models with MSE loss and early stopping based on validation loss.

The model's behavior is strongly influenced by its hyperparameters:

- **Sequence length:** controls the input time window. Longer sequences (e.g., 14) allow the model to

learn long-term dependencies but require more memory and may introduce noise.

- **Batch size:** impacts generalization and training time. Smaller batches (32–64) often lead to better generalization but increase training time. Larger batches (256–512) can converge faster but may overfit.
- **Learning rate:** governs how quickly the model adapts during training. We found 10^{-3} offered the best tradeoff between convergence speed and stability, aligning with prior hydrological studies [Kratzert et al.(2019)].
- **Hidden size:** affects representational capacity. Larger hidden states (64) increase expressiveness but may cause overfitting if not regularized.
- **Number of layers:** increases abstraction depth. Our experiments showed that 2 layers offered marginal gains over 1, consistent with diminishing returns seen in prior literature [Kratzert et al.(2019)].

We used various visualization techniques to evaluate the trained models, such as absolute error histograms, train vs validation loss line-plots, correlation heatmaps, and hyper-parameter comparisons through Box, Scatter, and Violin Plots.

2.1 Data & Preprocessing

Our primary dataset comes from NOAA's National Water level Observation Network (NWLON) system, spanning 217 stations (see Figure 6b) and over 127 million measurements, each taken at six-minute intervals. The full dataset totals over 82 Gigabytes. To ensure accuracy and reproducibility, we created Python scripts that automatically retrieved, validated, and stored the data for each station in TSV format, complete with timestamps, water-level readings, and quality flags.

After collection, we verified data completeness by cross-checking record counts and timestamps against NOAA metadata. We retained only validated entries, discarding provisional or missing data. Each station's dataset was then chronologically sorted and indexed to maintain temporal continuity.

For the selected focus site of Nawiliwili, Hawaii (Station 1611400), we recorded key metadata such as geographic coordinates, sensor depth, and observation intervals for model calibration. Figure 4 shows five different stations and their properties as stored in the dataset. We documented every download and preprocessing step in structured log files to guarantee full traceability. All data were stored on secure university servers and mirrored on the Chameleon Cloud environment for high-performance computation.

tidal	greatlakes	shefcode	state	timezone	timezonecorr	id	name	lat	lng	affiliations	tideType	established	removed	origyear	missing_percent
True	False	NWWH1	HI	HAST	-10	1611400	Nawiliwili	21.9544	-159.3561	NWLON	Mixed	1954-11-24 00:00:00.0	1991-02-16 00:00:00.0	1.6903284477206767e-06	
True	False	OOUH1	HI	HAST	-10	1612340	Honolulu	21.303333	-157.864528	NWLON	Mixed	1905-01-01 00:00:00.0	1989-01-20 00:00:00.0	0.0	
True	False	PRHH1	HI	HAST	-10	1612401	Pearl Harbor	21.3675	-157.963898	PORTS	Mixed	2023-05-31 00:00:00.0	2023-05-31 00:00:00.0		
True	False	MOKH1	HI	HAST	-10	1612480	Mokuoloe	21.433056	-157.79	NWLON	Mixed	1957-05-03 00:00:00.0	1989-01-21 00:00:00.0	0.0	
True	False	KLIH1	HI	HAST	-10	1615680	Kahului, Kahului Harbor	20.895	-156.469167	NWLON	Mixed	1946-12-19 00:00:00.0	1989-01-29 00:00:00.0	0.0	

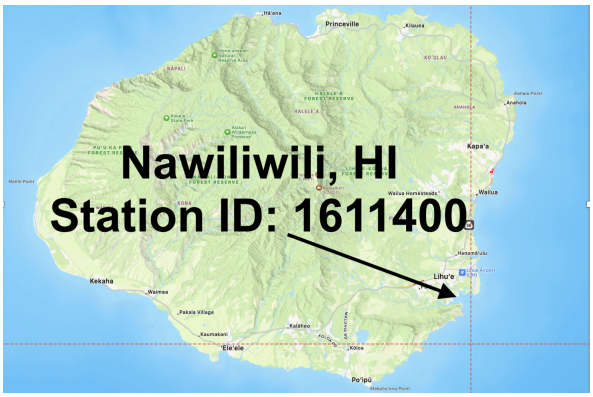
Figure 4: Sample data for metadata on water level stations; figure is created by student

Figure 5 shows the first nine entries in the final dataset.

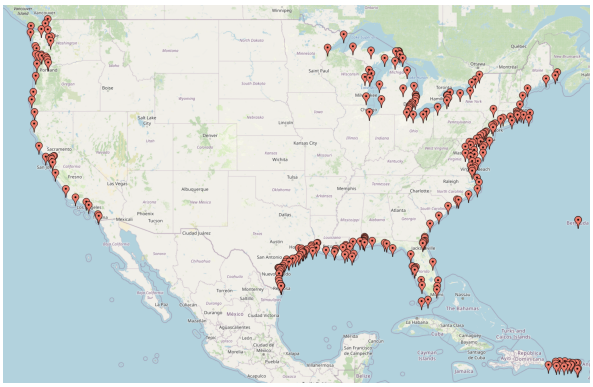
time	value	sigma	quality	inferred		flat		roc	threshold	station	datum
2018-01-01 00:00:00	0.273	0.002	v	0	0	0	0	1611400	mllw		
2018-01-01 00:06:00	0.278	0.003	v	0	0	0	0	1611400	mllw		
2018-01-01 00:12:00	0.277	0.003	v	0	0	0	0	1611400	mllw		
2018-01-01 00:18:00	0.276	0.006	v	0	0	0	0	1611400	mllw		
2018-01-01 00:24:00	0.28	0.004	v	0	0	0	0	1611400	mllw		
2018-01-01 00:30:00	0.28	0.002	v	0	0	0	0	1611400	mllw		
2018-01-01 00:36:00	0.271	0.002	v	0	0	0	0	1611400	mllw		
2018-01-01 00:42:00	0.279	0.002	v	0	0	0	0	1611400	mllw		
2018-01-01 00:48:00	0.282	0.003	v	0	0	0	0	1611400	mllw		

Figure 5: Sample data; figure is created by student

Our work focuses on the station in Nawiliwili, HI (Station ID: 1611400, see Figure 6a) due to the sensor data being complete with no missing values.



(a) NOAA's Nawiliwili station; graphic is extracted from Google Maps and edited with text overlay using the Photo application



(b) U.S. Map of Stations; graphic is created by student using Shapely/GeoPandas and an interactive Leaflet map

Figure 6: Target station and dataset coverage

2.2 Modeling Framework

We developed a modular pipeline in PyTorch for training and evaluating water-level forecasting models. After designing the computational framework, we implemented the full procedure using a combination of high-performance computing and modern machine learning tools. All modeling and data processing were executed in Python, primarily with PyTorch, NumPy, and Pandas. We wrote modular scripts to automate

each step, from dataset extraction and preprocessing to model training and evaluation, allowing for seamless scalability across different stations.

To handle the large dataset (over 82 GB), we used GPU-accelerated training on the Chameleon Cloud platform, leveraging an NVIDIA RTX 6000 GPU and 192 GB of RAM for efficient experimentation. We implemented early stopping, learning-rate scheduling, and cross-validation techniques to improve model stability and prevent overfitting. Each model was evaluated with Root Mean Squared Error (RMSE), Mean Absolute Error (MAE), and the Coefficient of Determination (R^2) metrics. RMSE measures the average magnitude of prediction errors, giving greater weight to large errors. MAE measures the average absolute difference between predicted and actual values, without squaring errors. R^2 quantifies how well the model's predictions explain the variability in the actual data. We use tools such as Plotly for visualization and parameter correlations.

Each model receives a fixed-length window of prior water levels as input and predicts future water levels at multiple time horizons (1, 3, 5, and 7 days). Input features are normalized per sequence using standard scaling. To understand how architectural design and input dimensionality influence performance, we explored three major experiments.

(1) Univariate Hyperparameter Sweep

We evaluated 120 configurations of a standard LSTM model, varying five key hyperparameters. Table 1 summarizes the grid search space. Each model was trained with mean squared error (MSE) loss and early stopping based on validation RMSE.

Table 1: Hyperparameter grid for univariate LSTM sweep; table is created by student

Parameter	**Values Tested**
Sequence Length	7, 14
Batch Size	32, 64, 128, 256, 512
Learning Rate	1x10^{-3}, 1x10^{-4}, 1x10^{-5}
Hidden Size	32, 64
Number of Layers	1, 2

(2) Model Architecture Comparison

To assess the effect of architecture choice, we fixed the best LSTM configuration and compared it against other recurrent models across 180 trials. Table 2 lists the architectures and configuration ranges. This experiment evaluated whether added architectural complexity led to meaningful gains in forecast accuracy

or generalization.

Table 2: Recurrent architectures tested in model comparison; table is created by student

Model Type	**Variants Tested**	**Parameter Ranges**
LSTM	Vanilla, BiLSTM	Hidden sizes: 32, 64
Conv-LSTM	1D convolution	Learning rates: 1x10^{-3}, 5x10^{-4}, 1x10^{-4}
GRU	Single, stacked	Layers: 1, 2
Attention-LSTM	Scaled dot-product	Batch sizes: 32, 64, 128

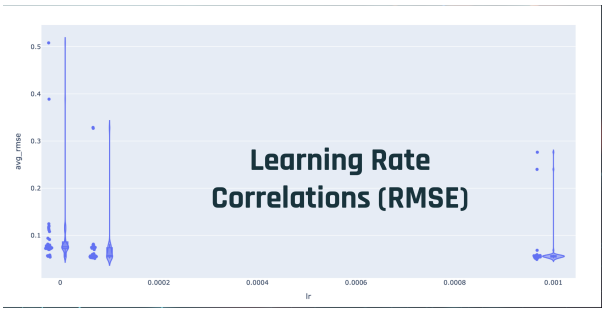
(3) Multivariate Input Extension

We extended the LSTM input space to include additional sources of temporal information. This aimed to determine whether richer inputs improve model performance or simply increase the risk of overfitting.

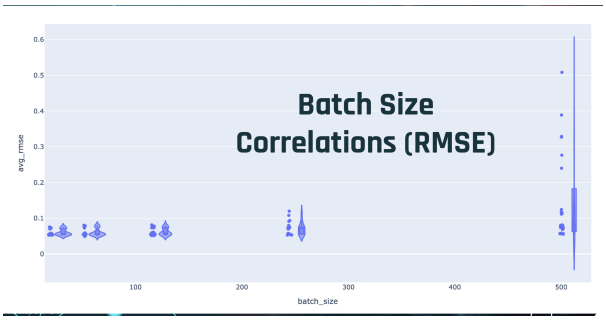
- Neighboring station water levels
- NOAA tidal predictions
- All 37 harmonic constituents from Nawiliwili

3 Performance Evaluation

Learning rate was the dominant factor. Models trained with 1×10^{-3} converged fastest and achieved the highest R^2, outperforming lower values by a wide margin (see Figure 7a). Batch size and number of layers had modest effects. Smaller batches (32–64) improved stability and generalization. One or two layers offered comparable results (see Figure 7b).



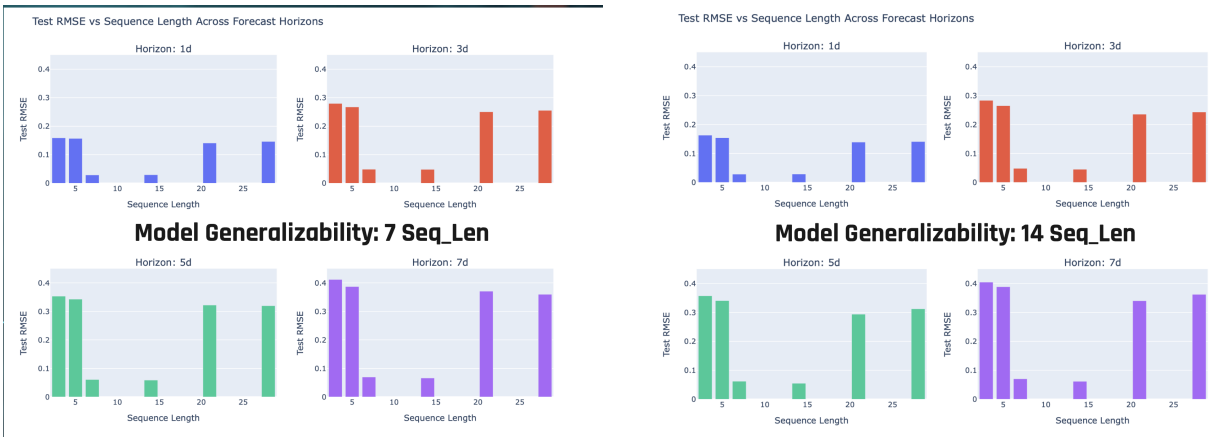
(a) Learning rate vs RMSE correlation



(b) Batch size vs RMSE correlation

Figure 7: Learning rate and batch size; figure is created by student with experimental data collected

Longer sequences (14 vs. 7) slightly improved accuracy but with a significant increase in training time. Models tuned for one seq length also did not generalize well for other sequence lengths (see Figure 8a and Figure 8b).



(a) Sequence Length of 7 Days

(b) Sequence Length of 14 Days

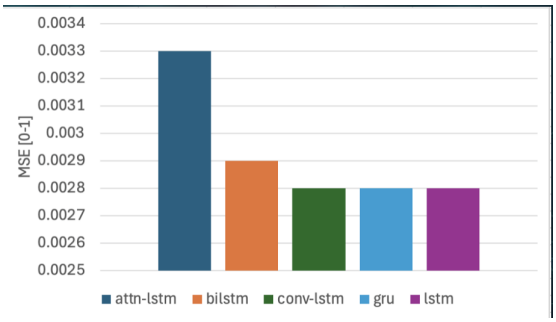
Figure 8: Model Generalizability; figure is created by student with experimental data collected

To visualize trends, we generated error histograms, loss curves, and correlation heatmaps using Python library Plotly, which allowed me to detect overfitting and assess convergence speed across model architectures.

Additional evaluation results and visualizations are provided in Appendix B.

3.1 Architecture Comparisons

Despite their theoretical advantages, BiLSTM, GRU, and Attention-based models did not outperform the tuned vanilla LSTM.



(a) MSE distribution across architectures

	BEST	Model	R^2	MSE	MAE	MaxAE	seq	batch	lr	hidden	layers
Horizon 1d	R^2	conv-lstm	0.980	0.001	0.022	0.133	7	64	0.0005	32	1
	MAE	conv-lstm	0.980	0.001	0.022	0.133	7	64	0.0005	32	1
	MaxAE	lstm	0.978	0.001	0.023	0.116	7	32	0.001	32	2
Horizon 3d	R^2	lstm	0.942	0.002	0.038	0.173	7	64	0.001	32	1
	MAE	conv-lstm	0.942	0.002	0.037	0.206	7	64	0.0005	32	1
	MaxAE	bilstm	0.940	0.002	0.039	0.169	7	32	0.0005	32	1
Horizon 5d	R^2	gru	0.910	0.004	0.047	0.251	7	64	0.0001	64	2
	MAE	lstm	0.910	0.004	0.047	0.227	7	128	0.0005	32	1
	MaxAE	gru	0.903	0.004	0.049	0.219	7	32	0.001	32	1
Horizon 7d	R^2	conv-lstm	0.891	0.004	0.053	0.251	7	64	0.001	32	1
	MAE	conv-lstm	0.891	0.004	0.053	0.251	7	64	0.001	32	1
	MaxAE	gru	0.880	0.005	0.056	0.222	7	32	0.001	32	1

(b) Forecast performance per horizon across architectures

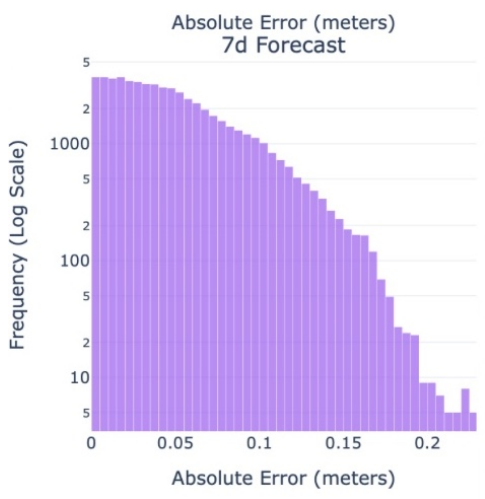
Figure 9: Architecture comparison; figure is created by student with experimental data collected

As shown in the figures above, variance was high, but performance distributions largely overlapped. This suggests that careful tuning matters more than exotic architecture selection.

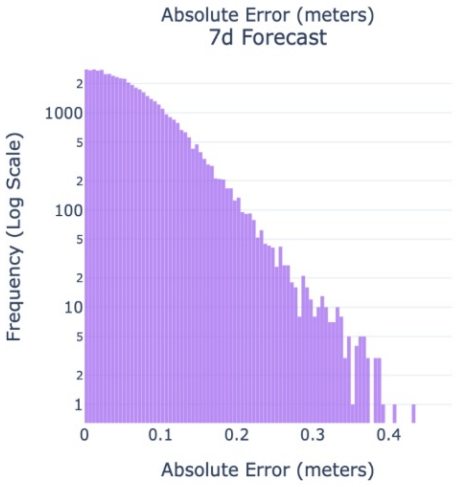
Additional evaluation results and visualizations are provided in Appendix B.

3.2 Multivariate Modeling

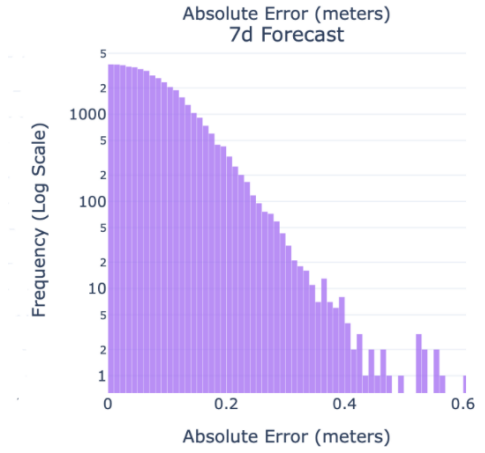
Contrary to expectations, adding auxiliary inputs hurt performance in most configurations. Multivariate models tended to overfit, especially when trained without retuning hyperparameters. Figure 10 shows how absolute error (maximum and distribution) increases for neighbor, NOAA, and constituent-based models compared to univariate LSTM.



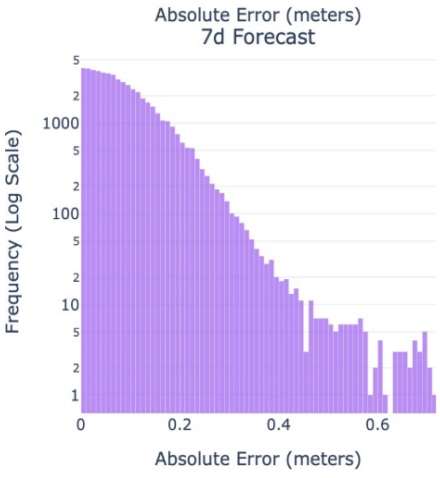
(a) Univariate LSTM (best configuration) - MaxAE: 0.241



(b) Multivariate with Neighbor Station - MaxAE: 0.445



(c) Multivariate with NOAA Predictions - MaxAE: 0.603



(d) Multivariate with 37 Constituents - MaxAE: 0.687

Figure 10: Absolute error histograms (log scale) across 7d horizons for various input types; figure is created by student with experimental data collected

Additional evaluation results and visualizations are provided in Appendix B.

3.3 LSTM Models Compared to NOAA Statistical Models

I performed statistical comparisons between model variants and NOAA's harmonic forecasts, demonstrating that optimized LSTM models can outperform prediction accuracy of traditional harmonic analysis between 2.1X and 4.7X (between 7D to 1D predictions). Figure 3 shows the histogram of all predictions using the LSTM compared to NOAA's predictions, showing the error size in meters.

Additional evaluation results and visualizations are provided in Appendix B.

3.4 Testbed Performance

To gauge real-world viability, we benchmarked training speed on two platforms. We trained models on both a local MacBook Pro (6-core Central Processing Unit, 16 GB RAM) and the Chameleon Cloud platform (24-core CPU, 192 GB RAM, NVIDIA RTX 6000 Graphics Processing Unit). On CPU, 2 epochs on 500,000 samples took 2 hours. On GPU, we trained 200 epochs in the same time, a 100× speedup. The hardware setup and components used in this work are illustrated in Figure 11.

(a) Chameleon Testbed (b) NVIDIA Quadro RTX 6000 (c) MacBook Pro

Figure 11: Hardware used to evaluate TidalMark. The setup includes the Chameleon Cloud testbed for distributed evaluation featuring a NVIDIA GPU to accelerate training and a MacBook Pro for early-stage development; these images were sourced from the Chameleon, NVIDIA, and Apple websites.

This gap highlights that GPU acceleration is essential for large-scale hyperparameter sweeps and production deployment.

4 Conclusion, Limitations, and Future Work

TidalMark demonstrates that well-tuned LSTM architectures outperform traditional harmonic-based forecasts in dynamic environments. Our results are proof of that: our properly tuned machine learning models consistently outperform the scientific-standard harmonic approaches between 2.1X and 4.7X (between 7D to 1D predictions). Figure 3 shows the histogram of all predictions using the LSTM compared to NOAA's predictions, showing the error size in meters.

These results confirmed our hypothesis that data-driven learning methods can adapt to nonlinear and non-stationary tidal behavior more effectively than fixed-parameter harmonic systems. The findings also demonstrated that model generalization is most influenced by learning rate and sequence length, providing key insights for future hydrological forecasting research. Others in the community agree with our findings that deep learning methods are increasingly surpassing classical statistical and physical models in accuracy and adaptability [Reichstein et al.(2019)]. Our experiences revealed that GPU acceleration is essential for large-scale environmental modeling, offering more than a 100× speedup over CPU training. This enables real-time adaptability, a crucial factor for flood-risk mitigation.

While TidalMark demonstrated the promise of deep learning for coastal water-level forecasting, several limitations affected the scope and reliability of the study. The most significant constraint was computational capacity. Training large neural networks on NOAA's 82-GB dataset required long runtimes and GPU access, which was limited to shared resources on the Chameleon Cloud. This reduced the number of full hyperparameter sweeps and long-range evaluations possible within the project timeline. Although NOAA's NWLON network provides high-quality six-minute interval data, some stations contained gaps, sensor drift, or incomplete calibration metadata. These inconsistencies introduced noise that may have affected model generalization. The project also relied solely on historical sea-level data, excluding external variables such as wind, pressure, and rainfall that could further enhance performance but at a higher computational cost.

Building on these findings, we plan to extend TidalMark into a full spatiotemporal Graph Neural Network framework, where each station is a node and edges represent geophysical relationships. This will allow the model to reason across space, not just time. We also aim to apply it to inland river systems. Additionally, we aim to explore automated hyperparameter optimization using Bayesian search. Finally, we plan to streamline the framework for production use, including edge deployment for early flood warnings.

Appendix A Source Code

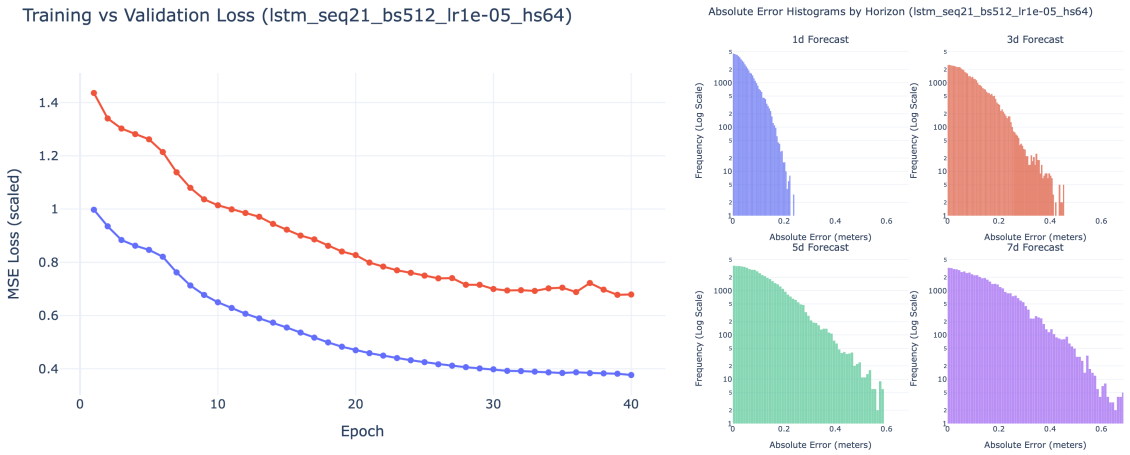
The collection of ten Python programs presented in Table 3 forms a comprehensive experimental framework for our data preprocessing, model training, and result analysis. Collectively comprising over 4K lines of code, these scripts employ core libraries such as `numpy`, `pandas`, `plotly`, `pytorch`, and `sklearn`. Together, they enable modular data preparation, multivariate LSTM training using harmonics, NOAA and neighbor-based features, and extensive hyperparameter sweep automation, as well as performance ranking and correlation visualization.

Table 3: Summary of Python programs used in the tidal forecasting framework.

Filename	**Description**
summary_organizer_sweeps.py	Aggregates and summarizes results from multiple hyperparameter sweep runs to identify best-performing configurations.
multivariate_lstm_constituents_modularized_revamped.py	Trains and evaluates LSTM models using tidal harmonic constituents as inputs to assess performance improvement.
multivariate_lstm_neighbor_modularized_revamped.py	Implements LSTM models incorporating data from neighboring stations to improve spatial prediction accuracy.
multivariate_lstm_noaa_modularized_revamped.py	Builds multivariate LSTM models combining NOAA tidal predictions and observed station data for improved temporal modeling.
train_sweep_modularized.py	Automates hyperparameter sweep training for LSTM-based machine learning models
split_noaa_predictions.py	Splits NOAA tidal prediction data into station-specific files for dataset organization
rank_best_configs_indiv.py	Ranks and analyzes top model configurations for individual experiments based on key performance metrics.
rank_best_configs_all.py	Compiles and compares best-performing configurations across all experiments
analyze_correlations_indiv.py	Computes and visualizes correlations between performance metrics and hyperparameters to reveal key relationships.
split_stations.py	Divides a master dataset into separate per-station files.

Appendix B Additional Evaluation Results

Figure 12 illustrates the training dynamics and absolute error distributions for the LSTM model configuration (`lstm_seq21_bs512_lr1e-05_hs64`) for a bad configuration. The left plot shows the convergence of training and validation loss with significant overfitting. The right plot visualizes error distributions across different forecast horizons.



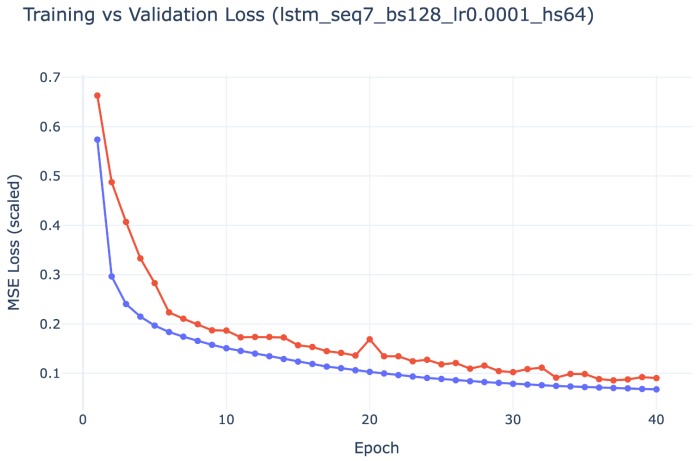
(a) Training vs. Validation Loss (b) Absolute Error Histograms by Horizon

Figure 12: Training performance and absolute error distributions for the LSTM model (`lstm_seq21_bs512_lr1e-05_hs64`). The left plot depicts training and validation loss curves, while the right plot shows horizon-wise error histograms on a logarithmic scale; these figures were generated by the student using Python and Plotly.

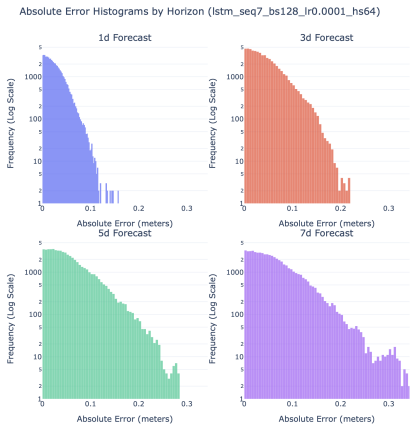
Figure 13 shows the training convergence and absolute error distribution for the well-performing LSTM model configuration (`lstm_seq7_bs128_lr0.0001_hs64`). Note how close the training and validation results are, indicating that the model was succesfully learning. The plots demonstrate stable loss reduction and tightly bounded forecast errors across all horizons.

Figure 14 compares the distribution of four key performance metrics: maximum absolute error (*max_ae*), mean squared error (*avg_mse*), coefficient of determination (*avg_r2*), and root mean squared error (*avg_rmse*) across varying batch sizes and learning rates for all trained models. Each violin plot illustrates the spread and central tendency of results, highlighting how training stability and predictive accuracy respond to these hyperparameters. The results suggest that moderate batch sizes and higher learning rates (around 0.001) consistently yield lower errors and stronger correlations, underscoring their importance for achieving balanced optimization and generalization performance in the overall model ensemble (Figure 14).

Figure 15 presents a comprehensive comparison of twelve LSTM configurations, each evaluated across



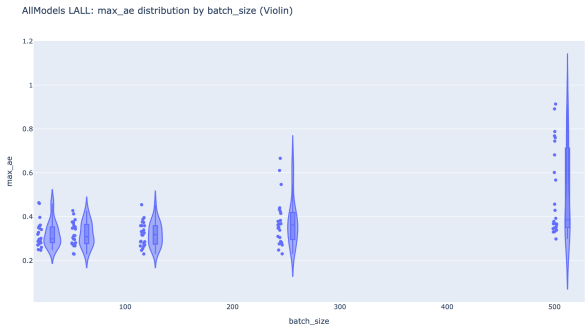
(a) Training vs. Validation Loss



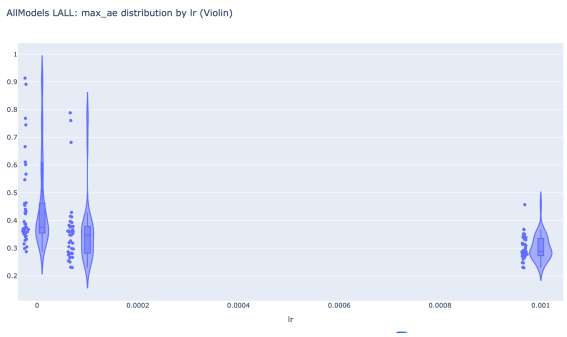
(b) Absolute Error Histograms by Horizon

Figure 13: Training performance and absolute error histograms for the LSTM configuration (`lstm_seq7_bs128_lr0.0001_hs64`). The left plot shows smooth convergence between training and validation losses, while the right plot depicts compact error distributions across 1–7 day forecasts, indicating model stability and low variance; these figures were generated by the student using Python and Plotly.

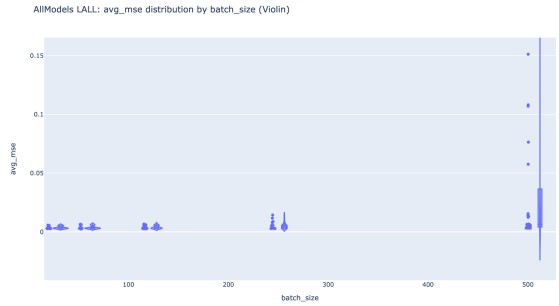
1-, 3-, 5-, and 7-day forecast horizons. The rows represent progressive increases in network capacity and architectural depth, while the columns contrast hidden size, batch size, and layer count variations. Overall, the results indicate that smaller and shallower models (top row) exhibit tighter and more stable error distributions, reflecting stronger short-horizon accuracy. In contrast, deeper and higher-capacity configurations (bottom row) demonstrate broader tails in their absolute error histograms, especially at 5- and 7-day horizons, signifying the trade-off between representational power and long-term generalization stability. This figure collectively illustrates how LSTM model complexity, training scale, and hyperparameter tuning influence forecast dispersion and reliability across temporal horizons.



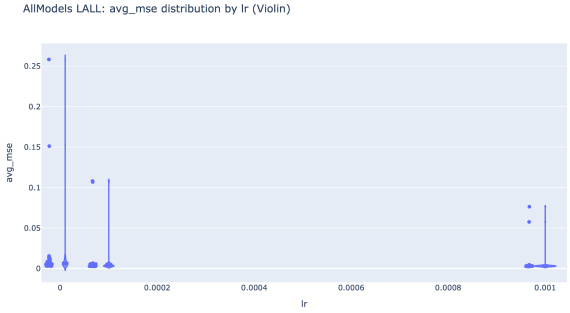
(a) **max_ae** by batch_size



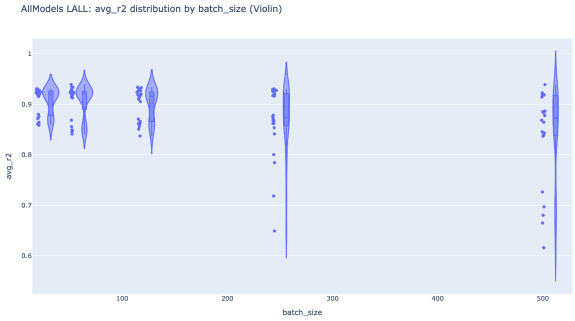
(b) **max_ae** by learning rate



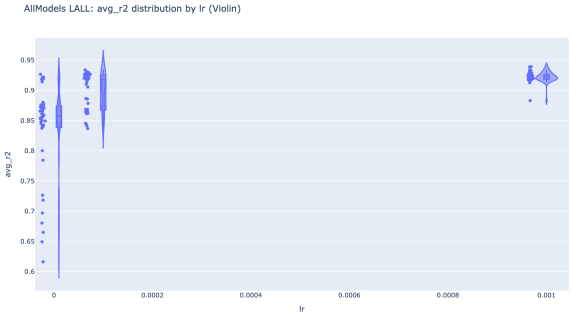
(c) **avg_mse** by batch_size



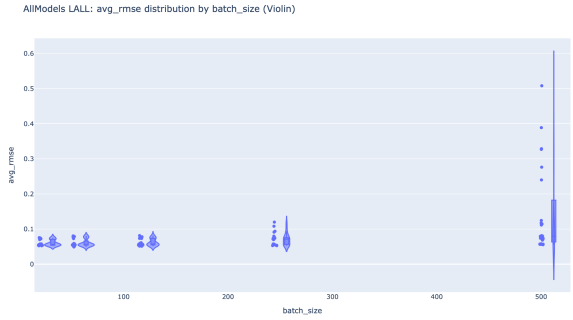
(d) **avg_mse** by learning rate



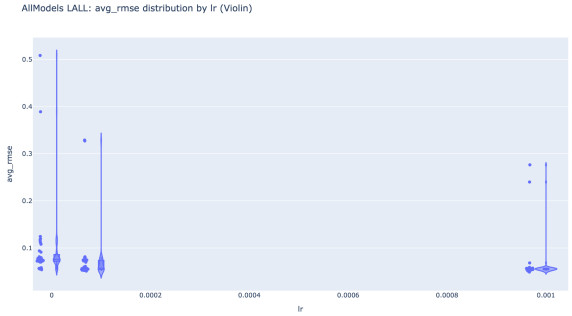
(e) **avg_r2** by batch_size



(f) **avg_r2** by learning rate



(g) **avg_rmse** by batch_size



(h) **avg_rmse** by learning rate

Figure 14: Violin plot comparisons of model performance metrics across batch size (left) and learning rate (right); these figures were generated by the student using Python and Plotly

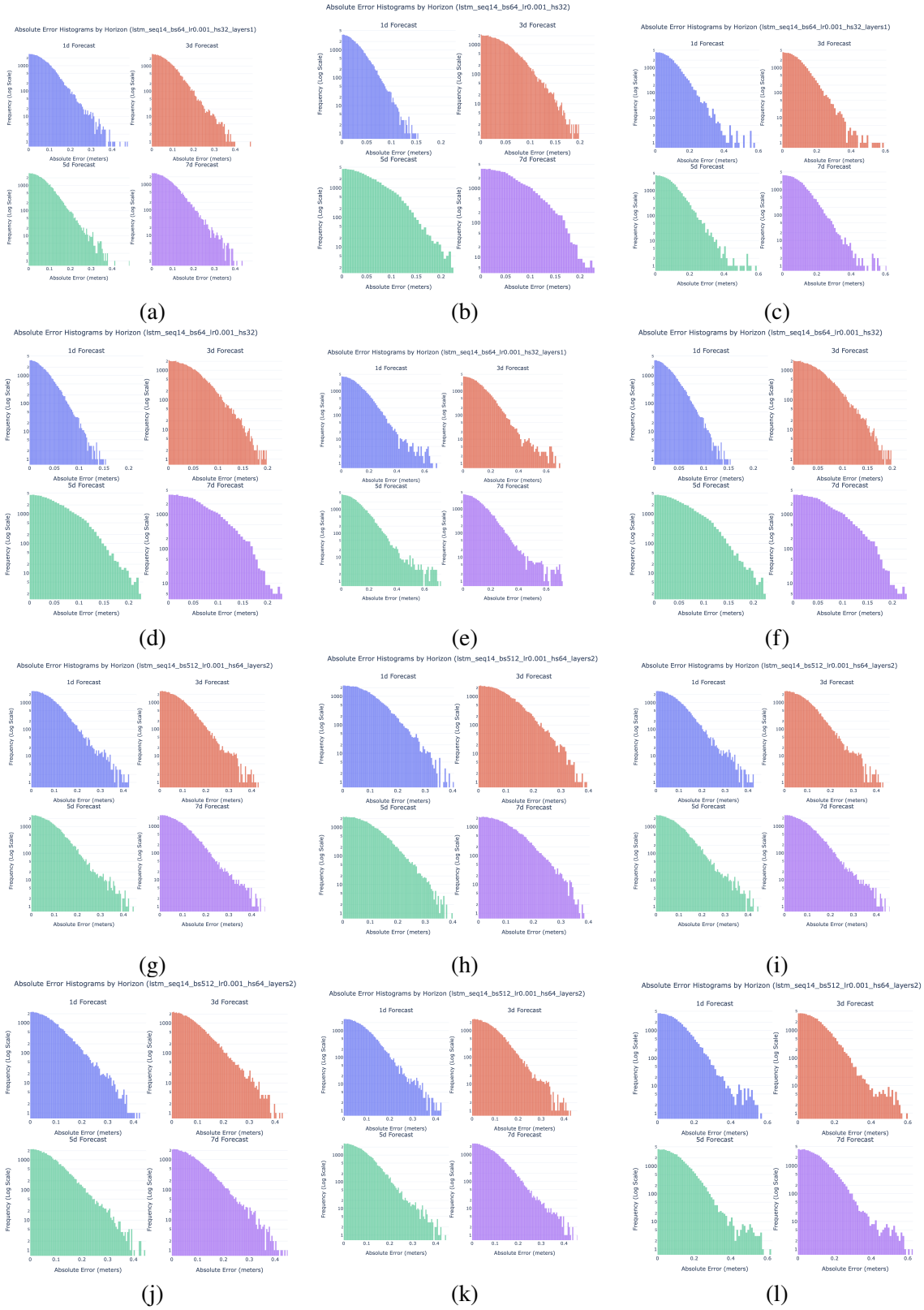
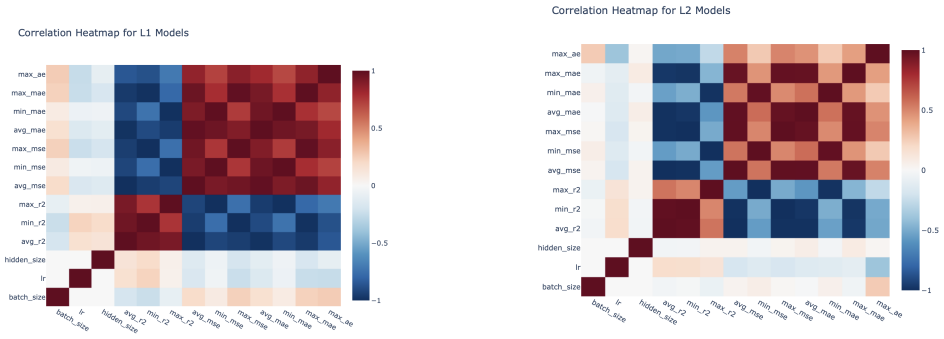


Figure 15: Comprehensive visualization of absolute error histograms across twelve LSTM configurations. Each plot shows frequency distributions (log scale) of forecast errors for 1-, 3-, 5-, and 7-day horizons under varying hidden sizes, layer counts, and batch configurations; these figures were generated by the student using Python and Plotly

Figure 16 compares how model performance changes when different weight-control methods are applied. The results show how limiting the size of network weights affects the relationships among accuracy metrics and helps stabilize model behavior.



(a) L1 Models (b) L2 Models

Figure 16: Correlation heatmaps for L1 and L2 models showing relationships between error metrics (MAE, MSE, R^2) and hyperparameters (learning rate, hidden size, batch size); strong red and blue regions indicate positive and negative correlations, respectively; these figures were generated by the student using Python and Plotly

Figure 17 compares the performance of the five neural architectures (Attention-LSTM, BiLSTM, Conv-LSTM, GRU, and LSTM) we've proposed. The results indicate that Conv-LSTM and GRU models generally achieve the best balance between accuracy and error reduction.

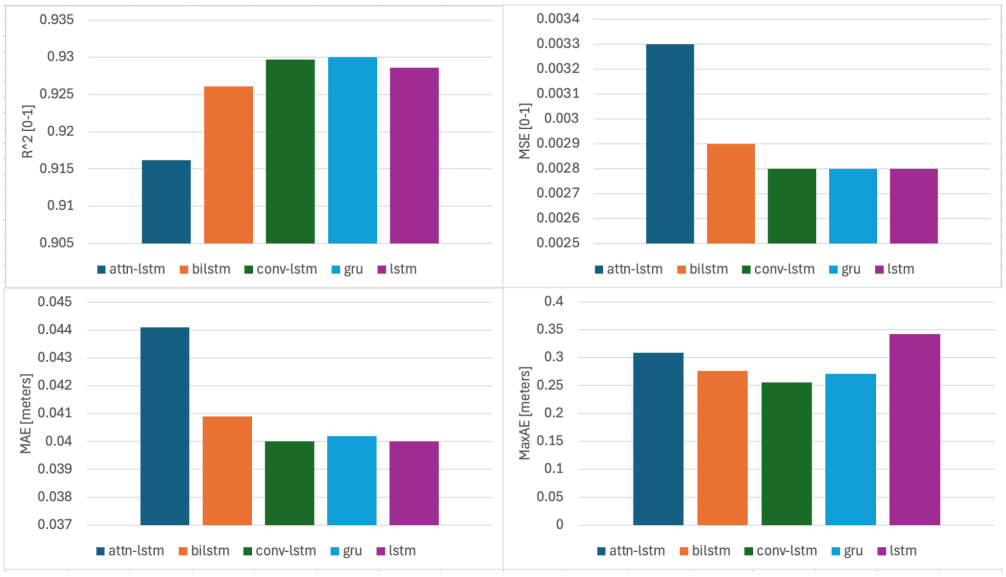


Figure 17: Comparison of neural network architectures based on R^2, MSE, MAE, and MaxAE metrics; the Conv-LSTM and GRU models show the strongest overall predictive performance, while Attention-LSTM exhibits higher error variability; these figures were generated by the student using Python and Plotly

Table 4 summarizes the LSTM model configurations and corresponding performance metrics across a wide range of hyperparameters. The results illustrate how changes in sequence length, batch size, and learning rate affect prediction accuracy and stability.

Table 4: Performance metrics for LSTM model configurations across varying hyperparameters. Each row lists results for different sequence lengths, batch sizes, learning rates, and hidden sizes, with corresponding evaluation metrics (R^2, MAE, RMSE, and MSE); these tables were generated by the student

model	seq	batch	lr	hidden	r2	mae	rmse	mse	run_dir
lstm	7	128	0.0001	64	0.913	0.044	0.056	0.003	lstm_seq7_bs128_lr0.0001_hs64
lstm	14	256	0.0001	64	0.879	0.051	0.066	0.005	lstm_seq14_bs256_lr0.0001_hs64
lstm	7	256	0.0001	64	0.875	0.052	0.067	0.005	lstm_seq7_bs256_lr0.0001_hs64
lstm	21	256	0.0001	64	0.866	0.054	0.069	0.005	lstm_seq21_bs256_lr0.0001_hs64
lstm	14	128	0.0001	64	0.862	0.055	0.070	0.005	lstm_seq14_bs128_lr0.0001_hs64
lstm	21	128	0.0001	64	0.859	0.055	0.071	0.006	lstm_seq21_bs128_lr0.0001_hs64
...	...	...	...	...	...	...	...	...	...
lstm	14	128	1.00E-06	64	0.505	0.105	0.130	0.020	lstm_seq14_bs128_lr1e-06_hs64
lstm	7	256	1.00E-06	64	0.456	0.112	0.138	0.022	lstm_seq7_bs256_lr1e-06_hs64
lstm	14	256	1.00E-06	64	0.426	0.114	0.141	0.023	lstm_seq14_bs256_lr1e-06_hs64
lstm	21	256	1.00E-06	64	0.422	0.114	0.142	0.023	lstm_seq21_bs256_lr1e-06_hs64
lstm	7	512	1.00E-06	64	0.336	0.125	0.157	0.026	lstm_seq7_bs512_lr1e-06_hs64
lstm	14	512	1.00E-06	64	0.299	0.131	0.164	0.028	lstm_seq14_bs512_lr1e-06_hs64

Table 5 compares the top ten overall models across architectures, highlighting configurations that achieved the highest R^2 and lowest error metrics. GRU and Conv-LSTM architectures emerge as the most reliable across multiple evaluation criteria.

Table 5: Comparison of the best-performing neural architectures with their associated hyperparameters and performance metrics; GRU and Conv-LSTM demonstrate robust generalization with high R^2 and low error values; these tables were generated by the student

Model	Layers	seq	batch	lr	hidden	layers	R^2	MSE	MAE	MaxAE
gru	2	7	64	0.0001	64	2	0.930	0.003	0.040	0.271
conv-lstm	1	7	64	0.001	32	1	0.930	0.003	0.040	0.256
lstm	1	7	128	0.0005	32	1	0.929	0.003	0.040	0.342
gru	1	7	32	0.0005	64	1	0.928	0.003	0.041	0.286
bilstm	1	7	32	0.001	32	1	0.926	0.003	0.041	0.276
conv-lstm	2	7	128	0.001	32	2	0.926	0.003	0.041	0.259
lstm	2	7	64	0.0005	32	2	0.924	0.003	0.042	0.273
bilstm	2	7	128	0.001	32	2	0.923	0.003	0.042	0.265
attn-lstm	1	7	32	0.001	64	1	0.916	0.003	0.044	0.309

References

[Consoli et al.(2014a)] Sergio Consoli, Diego Reforgiato Recupero, and Vanni Zavarella. 2014a. A survey on tidal analysis and forecasting methods for Tsunami detection. *arXiv preprint arXiv:1403.0135* (2014).

[Consoli et al.(2014b)] Sergio Consoli, Diego Reforgiato Recupero, and Vanni Zavarella. 2014b. A survey on tidal analysis and forecasting methods for Tsunami detection. *arXiv preprint arXiv:1403.0135* (2014). Traditional tidal forecasting methods based on harmonic analysis require long-term data and fail under non-astronomical influences such as weather.

[Hochreiter and Schmidhuber(1997a)] Sepp Hochreiter and Jürgen Schmidhuber. 1997a. Long Short-Term Memory. *Neural Computation* 9, 8 (1997), 1735–1780.

[Hochreiter and Schmidhuber(1997b)] Sepp Hochreiter and Jürgen Schmidhuber. 1997b. Long short-term memory. *Neural Computation* 9, 8 (1997), 1735–1780.

[Kratzert et al.(2019)] Frederik Kratzert, Daniel Klotz, Mathew Herrnegger, Andrew K Sampson, Sepp Hochreiter, and Grey Nearing. 2019. Towards learning universal, regional, and local hydrological behaviors via machine learning applied to large-sample datasets. *Hydrology and Earth System Sciences* 23, 12 (2019), 5089–5110.

[National Oceanic and Atmospheric Administration (NOAA)(2025)] National Oceanic and Atmospheric Administration (NOAA). 2025. *Station Selection – Tides Currents: Water Levels*. https://tidesandcurrents.noaa.gov/stations.html?type=Water+Levels Accessed: 2025-11-04.

[NOAA(2025)] NOAA. 2025. National Oceanic and Atmospheric Administration. https://www.noaa.gov/. Accessed: 2025-08-08.

[Perkins School for the Blind(2022)] Perkins School for the Blind. 2022. Sun – Moon – Tides graphic. Image file on Perkins.org. https://www.perkins.org/wp-content/uploads/2022/06/sun$_m oon_t ides_g raphic.jpg\, Accessed : 2025-11-04$.

[Pugh(2004)] David Pugh. 2004. *Tides, Surges and Mean Sea-Level* (2nd ed.). Wiley.

[Reichstein et al.(2019)] Markus Reichstein, Gustau Camps-Valls, Bjorn Stevens, Martin Jung, Joachim Denzler, Nuno Carvalhais, and Prabhat. 2019. Deep learning and process understanding for data-driven Earth system science. *Nature* 566, 7743 (2019), 195–204.

[Service(2024)] NOAA Ocean Service. 2024. What threats do coastal communities face? Online facts page. Lists threats including extreme natural events, sea level rise and coastal erosion..

[The Texas Tribune(2025)] The Texas Tribune. 2025. Texas Hill Country floods: What we know so far. *The Texas Tribune* (11 July 2025). https://www.texastribune.org/2025/07/11/texas-hill-country-floods-what-we-know/ Accessed: 2025-11-04.

[Wikimedia Commons contributors(2007)] Wikimedia Commons contributors. 2007. Tidal constituent sum. Image file on Wikimedia Commons. https://upload.wikimedia.org/wikipedia/commons/0/0e/Tidal$_c onstituent_s um.gif$ $Accessed$: $2025-11-04$.

[Wikipedia contributors(2025a)] Wikipedia contributors. 2025a. *Theory of tides*. https://en.wikipedia.org/wiki/Theory$_o f_t ides$ $Accessed$: $2025-11-04$.

[Wikipedia contributors(2025b)] Wikipedia contributors. 2025b. Tide — Harmonic analysis historical introduction by William Thomson. Wikipedia, accessed 2025-08-09. Describes development of harmonic analysis in the 1860s for tidal prediction..

[Wikipedia contributors(2025c)] Wikipedia contributors. 2025c. Tide — Harmonic analysis historical introduction by William Thomson. https://en.wikipedia.org/wiki/Theory$_o f_t ides.$ $Accessed$: $2025-08-09$.